

Comparator

- A **comparator** compares two string of bits to see whether they are equal to each other or other relation such as greater then or less then:
- This comparator compares the two bits and produces one of the 3 outputs as L ($A < B$), E ($A = B$) and G ($A > B$).
 - Example: if string $A = 0101$ and string $B = 0100$, then $A \neq B$.
 - 1 Bit Comparator(A, B are the part of single bit)
 - 2 Bit Comparator (A, B are the part of string or 2 bit)
 - 4 Bit Comparator (A, B are the part of string or 4 bit)



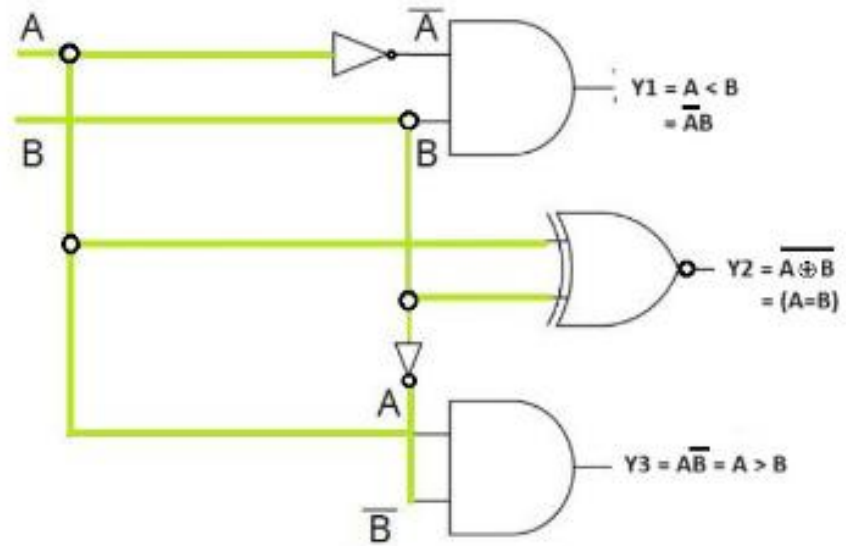
1-Bit Magnitude Comparator

INPUT S		OUTPUTS		
A	B	L (A<B)	E (A=B)	G (A>B)
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

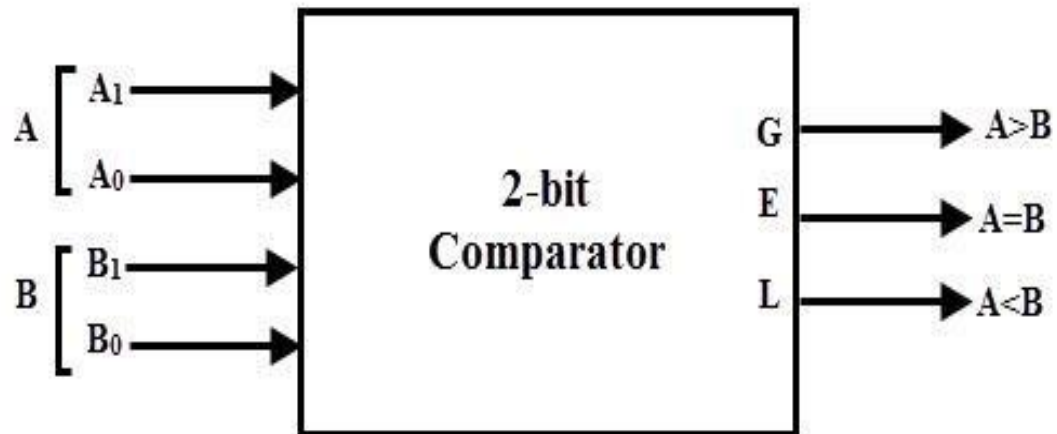
$$G = A > B = AB'$$

$$L = A < B = A'B$$

$$E = (A=B) = A'B' + AB$$



2-Bit Comparator



INPUT				OUTPUT		
A_1	A_0	B_1	B_0	L	E	G
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

2-Bit Magnitude Comparator

INPUT				OUTPUT		
A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

Truth Table of Output A>B

		A>B			
		B1B0	00	01	11
A1A0	00	0	0	0	0
	01	1	0	0	0
	11	1	1	0	1
	10	1	1	0	0

Truth Table of Output A=B

		A = B			
		B1B0	00	01	11
A1A0	00	1	0	0	0
	01	0	1	0	0
	11	0	0	1	0
	10	0	0	0	1

$$G = A>B: A1B1' + A0B1'B0' + A1A0B0'$$

$$E = (A=B) = A1'A0'B1'B0' + A1'A0B1'B0 + A1A0B1B0 + A1A0'B1B0'$$

$$E = (A0 \text{ Ex-Nor } B0) (A1 \text{ Ex-Nor } B1)$$

$$L = (A<B) = A1'B1 + A0'B1B0 + A1'A0'B0$$

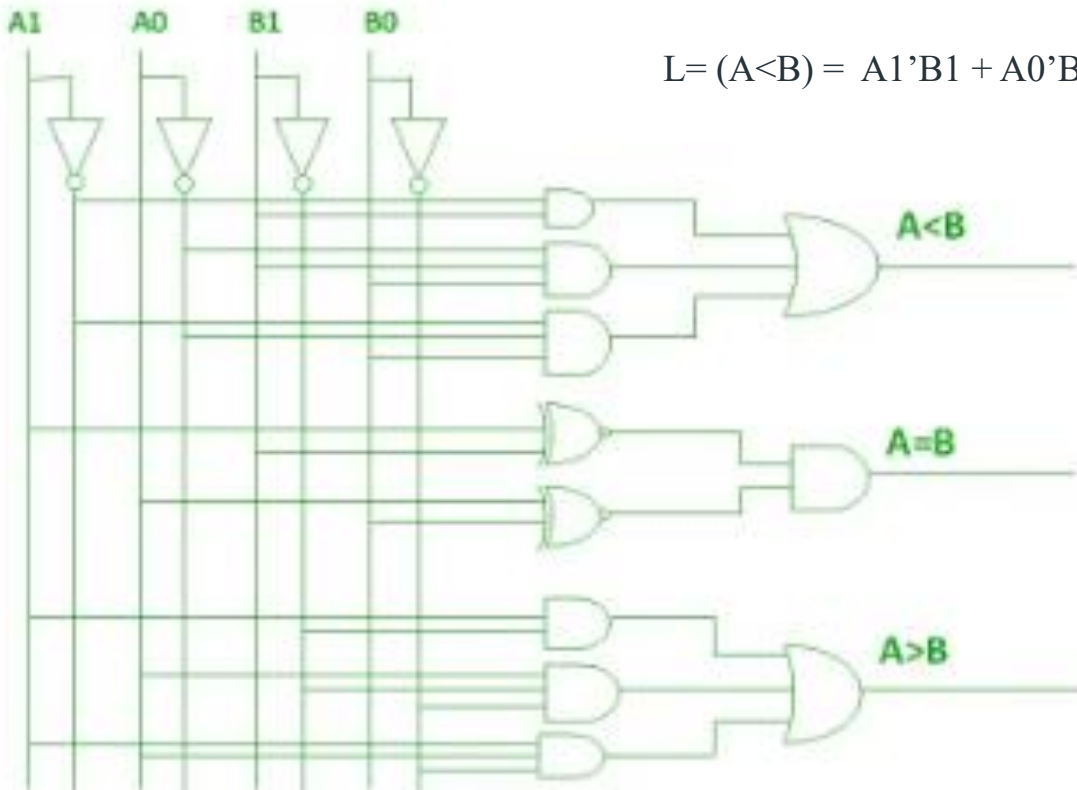
2-Bit Magnitude Comparator

$$G = A > B: A1B1' + A0B1'B0' + A1A0B0'$$

$$E = (A=B) = A1'A0'B1'B0' + A1'A0B1'B0 + A1A0B1B0 + A1A0'B1B0'$$

$$E = (A0 \text{ Ex-Nor } B0) (A1 \text{ Ex-Nor } B1)$$

$$L = (A < B) = A1'B1 + A0'B1B0 + A1'A0'B0$$



Applications of Comparators

- Comparators are used in central processing units ([CPUs](#)) and [microcontrollers](#) (MCUs).
- These are used in control applications in which the binary numbers representing physical variables such as temperature, position, etc. are compared with a reference value.
- Comparators are also used as process controllers and for Servo motor control.
- Used in password verification and biometric applications.

Advantage of Comparator

- Comparators are simple and efficient for comparison of binary values.
- Fast decision-making in Digital Circuits.
- This comparator can be easily integrated into complex systems like processors and arithmetic units.
- Design of comparator are modular, which allow them to scale solutions for comparing multi-bit numbers

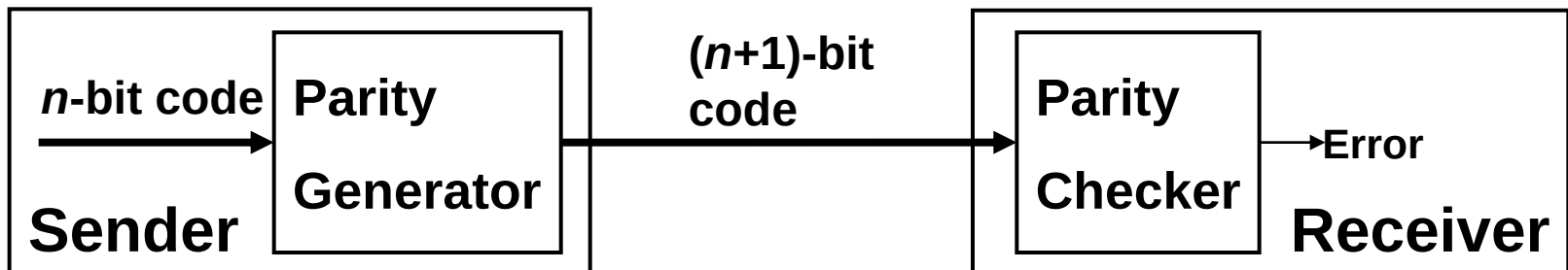
Disadvantages of Comparator

- Comparators have limit of bits for comparison.
- It requires more complex circuit for large bits.
- Power consumption increase with increase in the complexity of the circuit.

Parity

Parity Generation and Checking

- **Parity:**
 - The number of 1's in a bit stream.
- **Application:**
 - Error detection and correction.
- **Parity generator:**
 - the circuit that generates the parity bit before transmitting.
- **Parity checker:**
 - the circuit that checks the parity in the receiver.



What is Parity Generator?

- A Parity Generator is a Combinational Logic Circuit that Generates the Parity bit in the Transmitter.
- A Parity bit is used for the purpose of Detecting Errors during Transmissions of binary Information.
- It is an Extra bit included with a binary Message to Make the Number of 1's either Odd or Even.

Two Types of Parity

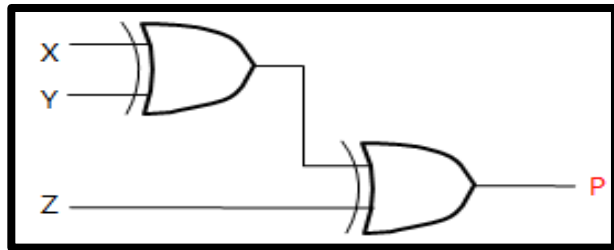
- In Even Parity, the added Parity bit will Make the Total Number of 1's an Even Amount.
- In Odd Parity, the added Parity bit will Make the Total Number of 1's an Odd Amount.

Parity Generator Truth Table

3-bit Message			Odd Parity Bit	Even Parity Bit
X	Y	Z		
0	0	0	1	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1

Boolean Expression

$$P(\text{EVEN}) = X \oplus (Y \oplus Z)$$



$$P(\text{ODD}) = X \odot (Y \oplus Z)$$

K-Map Simplification

X \ YZ				
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

X \ YZ				
	00	01	11	10
0	1	0	1	0
1	0	1	0	1

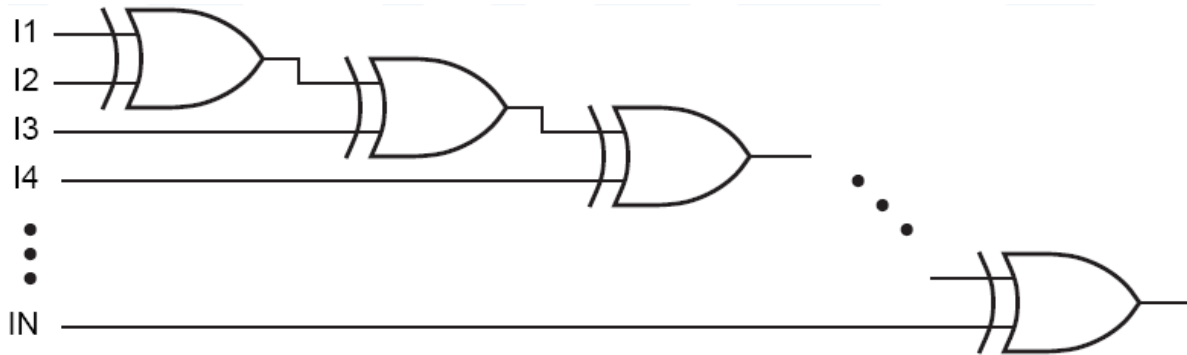
Parity Generation - Example

Even Parity Generator			
Three-Bit Message			Parity Bit
X	Y	Z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

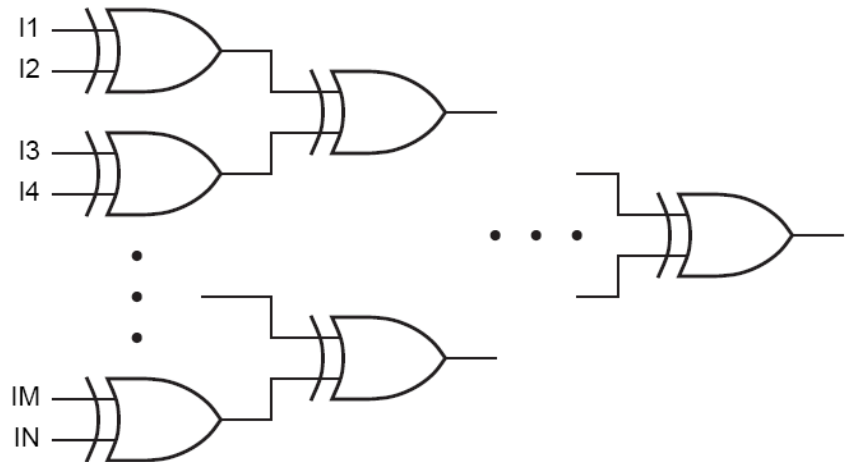
- $P(X,Y,Z)$ must produce a 1 for all the input combinations that contain an odd number of 1s
- Thus, it is a 3-input even function $P = X \oplus Y \oplus Z$

Parity Generator Circuits

- Daisy Chain

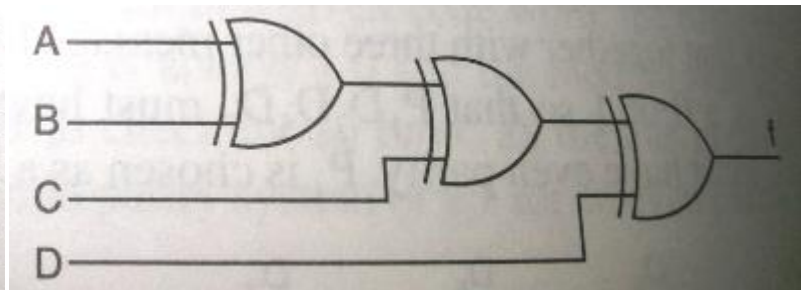
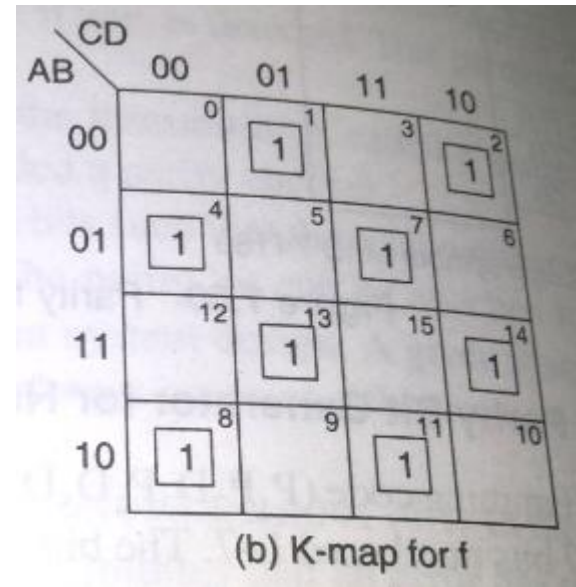


- Tree



EVEN PARITY GENERATOR

4-bit data input				Output parity bit (f)
A	B	C	D	
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

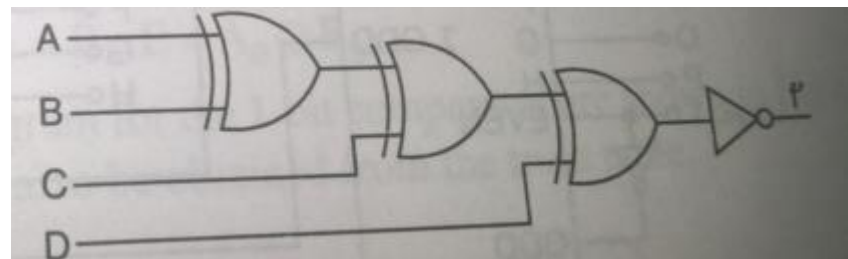


$$\begin{aligned}
 f &= \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}BCD + A\bar{B}\bar{C}D + AB\bar{C}\bar{D} + A\bar{B}C\bar{D} + AB\bar{C}D \\
 &= \bar{A}\bar{B}(C \oplus D) + \bar{A}B(\overline{C \oplus D}) + AB(C \oplus D) + A\bar{B}(\overline{C \oplus D}) \\
 &= (C \oplus D)(\bar{A} \oplus \bar{B}) + (\overline{C \oplus D})(A \oplus B) \\
 &= A \oplus B \oplus C \oplus D
 \end{aligned}$$

ODD PARITY GENERATOR

4-bit data input				Output parity bit $\bar{f}$
A	B	C	D	
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

CD \ AB				
	00	01	11	10
00	1 ⁰		1 ³	
01		1 ⁵		1 ⁶
11	1 ¹²		1 ¹⁵	
10		1 ⁸		1 ¹⁰



Odd and Even Functions

- The 1s of an **odd function** correspond to inputs with an **odd number of 1s (included output)**
- The **complement** of an odd function is called an **even function**
- The 1s of an **even function** correspond to inputs with an **even number of 1s**
- Implementation of odd and even functions use trees made up of 2-input XOR or XNOR gates

Parity Checker

- A Circuit that Checks the Parity in the Receiver is called Parity Checker.
- The Parity Checker Circuit Checks for Possible Errors in the Transmission.
- Since the Information Transmitted with Even Parity, the Received must have an even number of 1's. If it has odd number of 1's, it indicates that there is an Error occurred during Transmission.
- The Output of the Parity Checker is denoted by PEC (Parity Error Checker). If there is error, that is, if it has odd number of 1's, it will indicate 1. If not then PEC will indicate 0.

Even Parity Checker Truth Table

Decimal Equivalent	Four Bits Received				Parity Error
	P	A	B	C	PEC
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	0

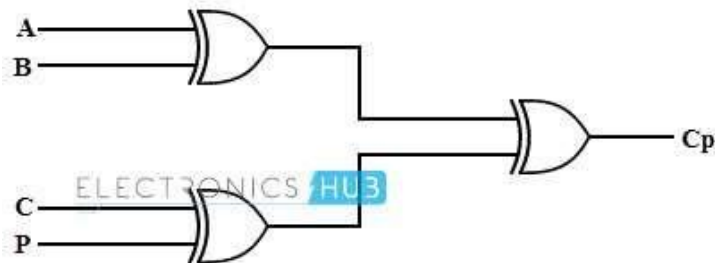
K-Map Simplification

PA \ BC	BC			
	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

Boolean Expression

$$PEC = (P \oplus A) \oplus (B \oplus C)$$

Logic Diagram

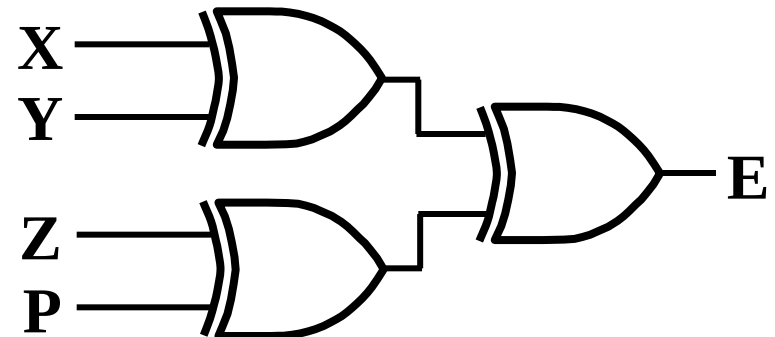
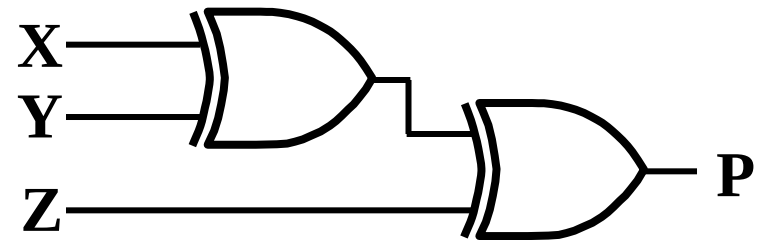


Parity Generators and Checkers

- A parity bit added to n -bit code produces $(n+1)$ -bit code with an odd (or even) count of 1s
- Odd Parity bit: count of 1s in $(n+1)$ -bit code is odd
 - So use an **even function** to generate the **odd parity bit**
- Even Parity bit: count of 1s in $(n+1)$ -bit code is even
 - So use an **odd function** to generate the **even parity bit**
- To check for odd parity
 - Use an **even function** to check the $(n+1)$ -bit code
- To check for even parity
 - Use an **odd function** to check the $(n+1)$ -bit code

Parity Generator & Checkers

- Design an even parity generator and checker for 3-bit codes
- Solution: Use 3-bit odd function to generate even parity bit
- Use 4-bit odd function to check for errors in even parity codes
- Operation: $(X,Y,Z) = (0,0,1)$ gives $(X,Y,Z,P) = (0,0,1,1)$ and $E = 0$
- If Y changes from 0 to 1 between generator and checker, then $E = 1$ indicates an error



Even Parity Checker

Table 4.17 5-bit even parity checker

5-bit message					Parity error
A	B	C	D	P	PE
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	0
0	0	1	0	0	1
0	0	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	1
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	1

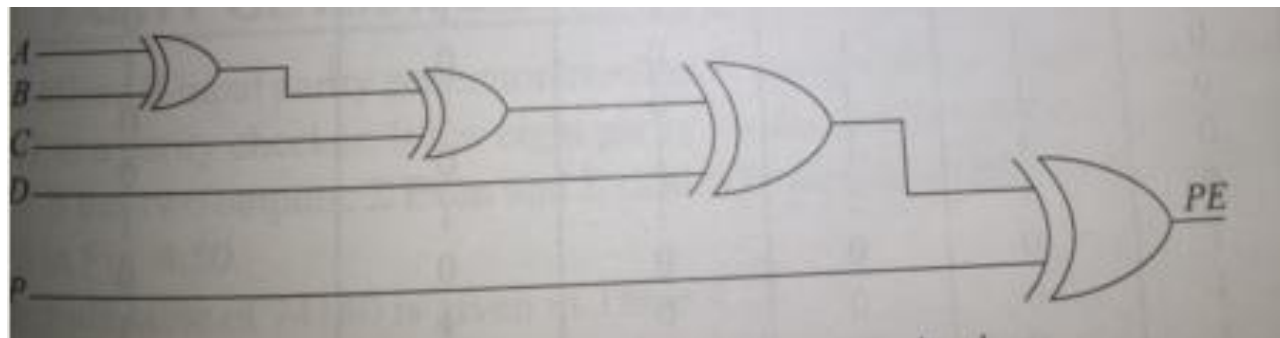
$A = 0$

BC	00	01	11	10
DP				
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

$A = 1$

BC	00	01	11	10
DP				
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

$$\begin{aligned}
 PE &= \overline{A}\overline{B}\overline{D}(C \oplus P) + \overline{A}\overline{B}D(C \odot P) + \overline{A}B\overline{D}(C \odot P) \\
 &\quad + \overline{A}BD(C \oplus P) + A\overline{B}\overline{D}(C \odot P) + A\overline{B}D(C \oplus P) \\
 &\quad + AB\overline{D}(C \oplus P) + ABD(C \odot P) \\
 &= \overline{A}\overline{B}[\overline{D}(C \oplus P) + D(\overline{C \oplus P})] + \overline{A}B[\overline{D}(C \oplus P) + D(\overline{C \oplus P})] \\
 &\quad + A\overline{B}[\overline{D}(C \oplus P) + D(\overline{C \oplus P})] + AB[\overline{D}(C \oplus P) + D(\overline{C \oplus P})] \\
 &= [(\overline{A}B + A\overline{B})(\overline{D}(C \oplus P))] + D(C + P) + (\overline{A}B + AB)(\overline{D}(C \oplus P) \\
 &\quad + D(\overline{C \oplus P}))((A \oplus B)(D \odot (C \oplus P)) + (A \odot B)(D \oplus C \oplus P)) \\
 &= [(A \oplus B)(\overline{D \oplus C \oplus P})] + [(\overline{A \oplus B})(D \oplus C \oplus P)] \\
 &= A \oplus B \oplus C \oplus D \oplus P
 \end{aligned}$$



ODD parity checker

5-bit message					Parity error
A	B	C	D	P	PE
0	0	0	0	0	1
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	1
0	0	1	1	1	0
0	1	0	0	0	0
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	0
1	0	1	0	0	1
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	1
1	1	1	1	0	1
1	1	1	1	1	0

$A = 1$

BC \ DP	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

$A = 0$

BC \ DP	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

